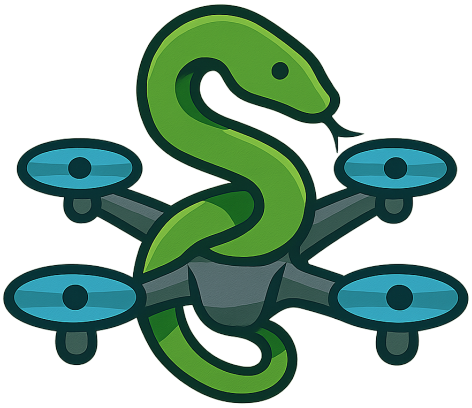
Technische Hochschule Ostwestfalen-Lippe

Bachelor of Science – Data Science

Software Design



Ausarbeitung zum Thema:

Python with Drones

Erstellt von:	Niclas Falke, Stefan Reitemeyer und Jonas Pottmeier
Matrikelnummer:	(Niclas),(Stefan) und 20116026 (Jonas)
Fachsemester:	3
Abgabedatum:	08.04.2026
Prüfer:	Prof. Dr.-Ing Rainer Rasche

Inhaltsverzeichnis

1 Zusammenfassung **1**

2 Einleitung **1**

3 Grundlagen **1**

3.1 Spielbasiertes Lernen und Computational Thinking 1

3.2 Browserbasierte Python-Ausführung 2

3.3 Verwendete Technologien 2

3.3.1 Next.js und TypeScript 3

3.3.2 Three.js 3

3.3.3 Pyodide 3

3.3.4 GitHub Actions und Pages 3

4 Methodik **4**

5 Ergebnisse **4**

6 Diskussion **4**

7 Schlussfolgerung **4**

8 Literaturverzeichnis **4**

9 Anhang **4**

1 Zusammenfassung

2 Einleitung

Die Vermittlung von Programmierkenntnissen stellt Lehrende und Lernende gleichermaßen vor grundlegende Herausforderungen. Abstrakte Konzepte wie Schleifen, Bedingungsanweisungen oder räumliches Denken sind für Einsteiger häufig schwer greifbar, da ein unmittelbares, visuelles Feedback fehlt. Traditionelle Lernumgebungen, in denen Code geschrieben und dessen Ausgabe ausschließlich in Textform erhalten wird, fördern zwar das syntaktische Verständnis, bieten jedoch wenig Motivation für Anfänger.

Unser Projekt *PythonWithDrones* soll dies adressieren, indem es Programmieren mit einem interaktiven, spielerischen Element verbindet. Lernende schreiben Python-Code, um eine virtuelle Drohne durch verschiedene Level zu steuern. Die Drohne muss dabei Hindernisse umfliegen und ein Zielportal erreichen. Der Drohnenzustand wird in Echtzeit in einer dreidimensionalen Visualisierung im Browser dargestellt, wodurch abstrakte Programmierkonzepte in eine konkret erfahrbare, räumliche Welt übertragen werden.

Die Grundidee des Projekts gründet in der Überzeugung, dass Programmieren für jeden erlernbar ist und ähnliche wie das Erlernen einer natürlichen Sprache durch Wiederholung und Übung gefestigt wird. Durch unmittelbares Feedback und schrittweise steigende Schwierigkeitsgrade soll der Einstieg in die Programmierung ermöglicht und erleichtert werden. Das Projekt wurde im Rahmen des Hochschulmoduls *Software Design* entwickelt und demonstriert dabei sowohl einen pädagogischen Ansatz als auch moderne Softwarearchitekturprinzipien. Die vollständig browserbasierte Anwendung ist ohne jegliche lokale Installation unter `https://pottmeier.github.io/PythonWithDrones/` erreichbar.

3 Grundlagen

Dieses Kapitel legt die konzeptionellen und technischen Grundlagen dar, auf denen *PythonWithDrones* aufbaut. Zunächst werden pädagogische Konzepte des spielbasierten Lernens erläutert, anschließend die eingesetzten Technologien beschrieben.

3.1 Spielbasiertes Lernen und Computational Thinking

Spielbasiertes Lernen nutzt Spielmechaniken gezielt, um Lernziele zu erreichen und die intrinsische Motivation der Lernenden zu steigern. Für das Erlernen von Programmiersprachen ist dieser Ansatz besonders geeignet. Durch sofortiges visuelles Feedback beobachten Lernende die Auswirkungen ihres Codes direkt und können ein intuitives Verständnis von Schleifen, Bedingungen und Ablaufsteuerung aufbauen. Das Levelsystem von *PythonWithDrones* folgt dabei dem Prinzip des schrittweisen Aufbaus, bei dem Lernende sukzessive an komplexere Aufgaben herangeführt werden. Einfache Level erfordern lediglich grundlegende Bewegungsbefehle, während fortgeschrittene Level den Einsatz von Schleifen und Bedingungsanweisungen voraussetzen.

Darüber hinaus fördert das Projekt Fähigkeiten des strukturierten, algorithmischen Denkens.

Das Navigieren einer Drohne durch einen Hindernisparcours erfordert die Zerlegung des Gesamtproblems in einzelne Teilschritte, das Erkennen von Bewegungsmustern sowie die präzise Formulierung eines Algorithmus in Python-Syntax. Diese Kompetenzen sind grundlegend für das Programmieren und bilden eine solide Basis für weiterführende Themen der Informatik.

3.2 Browserbasierte Python-Ausführung

Traditionell setzt die Ausführung von Python-Code eine lokale Interpreter-Installation voraus, was für Lernumgebungen eine erhebliche Einstiegshürde darstellt. Moderne Technologien ermöglichen es jedoch, Python direkt im Browser auszuführen, ohne serverseitige Infrastruktur zu benötigen. Grundlage hierfür ist WebAssembly, ein binäres Instruktionsformat, das in modernen Browsern nahezu nativ ausgeführt werden kann. Auf dieser Basis ermöglicht Pyodide die vollständige Ausführung des CPython-Interpreters im Browser.

PythonWithDrones nutzt diesen Ansatz konsequent. Die gesamte Anwendung ist als statische Webseite deployt und benötigt keinerlei Backend. Lernende benötigen ausschließlich einen modernen Browser, um sofort mit dem Programmieren zu beginnen. Wir haben diese Entscheidung getroffen um es allen Studierenden der Hochschule zu ermöglichen unser Projekt zu nutzen.

3.3 Verwendete Technologien

Die Implementierung basiert auf einem vollständig browserbasierten Technologie-Stack ohne serverseitige Infrastruktur. Tabelle 1 gibt einen Überblick über die eingesetzten Technologien, anschließend werden die Kernkomponenten näher erläutert.

Table 1: Technologie-Stack von *PythonWithDrones*

Technologie	**Kategorie**	**Funktion im System**
Next.js	Framework	UI-Routing, SSG, Projektstruktur
TypeScript	Sprache	Typsichere Implementierung, statische Typprüfung
Three.js	Bibliothek	3D-Visualisierung der Drohne und Level
Pyodide	Wasm-Runtime	Python-Ausführung im Browser
Python	Lernsprache	Drohnensteuerung via Befehls-API
CSS	Styling	Layout und visuelle Gestaltung
GitHub Actions	CI/CD	Automatisierter Build und Deployment
GitHub Pages	Hosting	Statisches, serverfreies Deployment

3.3.1 Next.js und TypeScript

Next.js ist ein auf React basierendes Framework, das dateibasiertes Routing, Static Site Generation sowie eine klare Projektstruktur mitbringt. Als typisierte Obermenge von JavaScript ermöglicht TypeScript statische Typprüfung bereits zur Kompilierzeit, was besonders für die Definition von Datenstrukturen wie dem Drohnenzustand (`DroneState`) sowie der Schnittstelle zwischen Pyodide und der 3D-Engine von Vorteil ist. Die Entscheidung für Next.js ermöglicht es zudem, die Anwendung als vollständig statischen Export zu bauen und direkt über GitHub Pages auszuliefern, ohne einen eigenen Server betreiben zu müssen. Komponenten für Editor, 3D-Viewport und Level-Selektor sind als eigenständige React-Komponenten implementiert und können unabhängig voneinander entwickelt und getestet werden.

3.3.2 Three.js

Three.js abstrahiert die WebGL-API und bietet eine deklarative Schnittstelle zur Erstellung von 3D-Szenen im Browser. Konzepte wie Szene, Kamera, Beleuchtung und Geometrien entsprechen denen professioneller 3D-Engines und ermöglichen die räumliche Darstellung komplexer Welten. In *PythonWithDrones* wird Three.js eingesetzt, um Drohne, Hindernisse und Zielportal dreidimensional darzustellen. Die Drohnenbewegung wird über einen Zustandsvektor aus Position und Rotation gesteuert, den der Python-Interpreter bei jeder Befehlsausführung aktualisiert, sodass Lernende eine unmittelbare visuelle Rückmeldung auf ihren Code erhalten.

3.3.3 Pyodide

Pyodide kompiliert den CPython-Interpreter zu WebAssembly und ermöglicht damit die Ausführung von Python vollständig im Browser, ohne serverseitige Infrastruktur zu benötigen. In *PythonWithDrones* bildet Pyodide das architektonische Herzstück. Der von Lernenden geschriebene Code wird über die Pyodide-API ausgeführt, wobei Drohnenbefehle als Python-Funktionen wie etwa `move()`, `turn_left()` oder `turn_right()` in den Python-Namespace injiziert werden. Jeder Funktionsaufruf übersetzt sich in eine Zustandsänderung des Drohnenobjekts, die Three.js anschließend visualisiert. Diese Brücke zwischen Python und JavaScript bildet das Kernstück der gesamten Anwendung.

3.3.4 GitHub Actions und Pages

Der gesamte Build- und Deploymentprozess wird über *GitHub Actions* automatisiert. Bei jedem Push auf den `main`-Branch wird eine Pipeline ausgelöst, die den TypeScript-Code kompiliert, den statischen Next.js-Export erzeugt und auf *GitHub Pages* veröffentlicht. Diese Kombination ermöglicht uns ein vollständig serverfreies und kostenloses Deployment.

4 Methodik

5 Ergebnisse

6 Diskussion

7 Schlussfolgerung

8 Literaturverzeichnis

References

9 Anhang